

Ex. No. 1.a

Implementation of Constructors in Java

Aim

To write a Java program to demonstrate the use of various constructors and constructor overloading in Java.

Algorithm

1. Start the program.
2. Create a class named Student.
3. Declare instance variables name, rollNo, and department.
4. Define a no-argument constructor to initialize default values.
5. Define a parameterized constructor to initialize all data members.
6. Define an overloaded constructor that initializes only the name and roll number.
7. Define a copy constructor that copies the values from another Student object.
8. Create a display() method to print the object details.
9. In the main() method:
 - Create an object using the no-argument constructor.
 - Create an object using the parameterized constructor.
 - Create an object using the overloaded constructor.
 - Create another object using the copy constructor.
10. Display the details of all objects.
11. Stop the program.

Program

```
class Student
{
    String name;
    int rollNo;
    String department;

    Student()
    {
        name = "Unknown";
        rollNo = 0;
        department = "Not Assigned";
    }
    Student(String name, int rollNo, String department)
    {
        this.name = name;
        this.rollNo = rollNo;
        this.department = department;
    }
    Student(String name, int rollNo)
    {
        this.name = name;
        this.rollNo = rollNo;
        this.department = "CSE";
    }
    Student(Student s)
    {
        this.name = s.name;
```

```

        this.rollNo = s.rollNo;
        this.department = s.department;
    }
    void display()
    {
        System.out.println("Name      : " + name);
        System.out.println("Roll No   : " + rollNo);
        System.out.println("Department : " + department);
        System.out.println();
    }
}
public class ConstructorDemo
{
    public static void main(String args[])
    {
        Student s1 = new Student();
        Student s2 = new Student("Rahul",101,"CSE");
        Student s3 = new Student("Priya",102);
        Student s4 = new Student(s2);
        s1.display();
        s2.display();
        s3.display();
        s4.display();
    }
}

```

Output

Name : Unknown
 Roll No : 0
 Department : Not Assigned

Name : Rahul
 Roll No : 101
 Department : CSE

Name : Priya
 Roll No : 102
 Department : CSE

Name : Rahul
 Roll No : 101
 Department : CSE

Result

Thus, the Java program to demonstrate the implementation of various was executed successfully.

Ex. No. 1.b Implementation of Static Variables and Static Methods

Aim

To write a Java program to demonstrate the use of **static variables** and **static methods**.

Algorithm

1. Start the program.
2. Create a class named Employee.
3. Declare instance variables name and id.
4. Declare a static variable company.
5. Create a constructor to initialize employee details.
6. Create a static method changeCompany() to modify the company name.
7. Create a method display() to print employee details.
8. In the main() method:
 - o Create two employee objects.
 - o Display their details.
 - o Change the company name using the static method.
 - o Display the updated details.
9. Stop the program.

Program

```
class Employee
{
    int id;
    String name;
    static String company = "ABC Technologies";

    Employee(int i, String n)
    {
        id = i;
        name = n;
    }

    static void changeCompany(String cname)
    {
        company = cname;
    }

    void display()
    {
        System.out.println(id + " " + name + " " + company);
    }
}

public class StaticMemberDemo
{
    public static void main(String args[])
    {
        Employee e1 = new Employee(101, "Anand");
        Employee e2 = new Employee(102, "Priya");

        System.out.println("Before Changing Company Name");
```

```
e1.display();
e2.display();

Employee.changeCompany("XYZ Solutions");

System.out.println("\nAfter Changing Company Name");
e1.display();
e2.display();
}
}
```

Output

Before Changing Company Name
101 Anand ABC Technologies
102 Priya ABC Technologies

After Changing Company Name
101 Anand XYZ Solutions
102 Priya XYZ Solutions

Result

Thus, the Java program to demonstrate the use of **static variables** and **static methods** was executed successfully.

Ex. No. 2.a Implementation of Single Inheritance

Aim

To write a Java program to demonstrate **single inheritance** by inheriting the properties and methods of a base class into a derived class.

Algorithm

1. Start the program.
2. Create a base class named Person with data members name and age.
3. Define a constructor in the Person class to initialize the data members.
4. Create a member function displayPerson() to display the person's details.
5. Create a derived class named Student that extends the Person class.
6. Add a data member rollNo to the Student class.
7. Define a constructor in the Student class that invokes the superclass constructor using the super keyword.
8. Create a member function displayStudent() to display the student's details.
9. In the main() method, create an object of the Student class.
10. Call the methods to display the inherited and derived class data.
11. Stop the program.

Program

```
class Person
{
    String name;
    int age;

    Person(String name, int age)
    {
        this.name = name;
        this.age = age;
    }

    void displayPerson()
    {
        System.out.println("Name : " + name);
        System.out.println("Age : " + age);
    }
}

class Student extends Person
{
    int rollNo;

    Student(String name, int age, int rollNo)
    {
        super(name, age);
        this.rollNo = rollNo;
    }

    void displayStudent()
    {
```

```
        System.out.println("Roll No : " + rollNo);
    }
}

public class InheritanceDemo
{
    public static void main(String args[])
    {
        Student s = new Student("Rahul", 20, 101);

        System.out.println("Student Details");
        s.displayPerson();
        s.displayStudent();
    }
}
```

Output

```
Student Details
Name : Rahul
Age : 20
Roll No : 101
```

Result

Thus, the Java program to demonstrate **single inheritance** by inheriting the properties and methods of a super class into a sub class was executed successfully.

Ex. No. 2.b Run-Time Polymorphism Using Method Overriding

Aim

To write and execute a Java program to demonstrate run-time polymorphism using method overriding, where a superclass reference is used to refer to objects of different subclasses.

Algorithm

1. Start the program.
2. Define a superclass Figure with two data members, dim1 and dim2.
3. Define a constructor in Figure to initialize the dimensions.
4. Define an area() method in the Figure class that returns 0 and displays that the area is undefined.
5. Create a subclass Rectangle that extends Figure.
6. Use the super() keyword in the Rectangle constructor to initialize the superclass members.
7. Override the area() method in Rectangle to calculate the area of the rectangle.
8. Create another subclass Triangle that extends Figure.
9. Use super() in the Triangle constructor and override the area() method to calculate the area of a right triangle.
10. Create a reference variable f of type Figure.
11. Assign the Rectangle object to the Figure reference and call area().
12. Assign the Triangle object to the same Figure reference and call area().
13. Assign a Figure object to the reference and call area().
14. Observe that the appropriate overridden method is selected at run time.
15. Stop the program.

Program

```
class Figure
{
    double dim1;
    double dim2;

    Figure(double a, double b) {
        dim1 = a;
        dim2 = b;
    }

    double area() {
        System.out.println("Area for Figure is undefined.");
        return 0;
    }
}

class Rectangle extends Figure {

    Rectangle(double a, double b) {
        super(a, b);
    }

    double area() {
        System.out.println("Inside Area for Rectangle.");
    }
}
```

```

        return dim1 * dim2;
    }
}

class Triangle extends Figure {

    Triangle(double a, double b) {
        super(a, b);
    }

    // Override area for right triangle
    double area() {
        System.out.println("Inside Area for Triangle.");
        return dim1 * dim2 / 2;
    }
}

public class FindAreas {

    public static void main(String[] args) {

        Figure f = new Figure(10, 10);
        Rectangle r = new Rectangle(9, 5);
        Triangle t = new Triangle(10, 8);

        // Figure reference refers to Rectangle object
        f = r;
        System.out.println("Area is " + f.area());

        // Figure reference refers to Triangle object
        f = t;
        System.out.println("Area is " + f.area());

        // Figure reference refers to Figure object
        f = new Figure(10, 10);
        System.out.println("Area is " + f.area());
    }
}

```

Sample Output

```

Inside Area for Rectangle.
Area is 45.0
Inside Area for Triangle.
Area is 40.0
Area for Figure is undefined.
Area is 0.0

```

Result

Thus, the Java program to demonstrate run-time polymorphism using method overriding was successfully executed.

Ex.no:3.a

User Defined Package

Aim

To create a **user-defined package** in Java, define a class with different access specifiers, import the package into another program, and demonstrate how access specifiers (**public, protected, default, and private**) control the accessibility of class members from outside the package.

Algorithm

Step 1: Start the program.

Step 2: Create a user-defined package named **mypackage**.

Step 3: Create a class named **Student** inside the package.

Step 4: Declare the data members:

- name with default access.
- age with protected access.
- department with public access.
- cgpa with private access.

Step 5: Define a **parameterized constructor** to initialize the data members (name, age, department, and cgpa) using the values passed as arguments.

Step 6: Create a public method named **display()** to display all the student details.

Step 7: Compile the Student.java file using:

```
javac -d . Student.java
```

Step 8: Create another class named **PackageDemo** outside the package.

Step 9: Import the Student class using the import statement.

Step 10: Create a Scanner object to read input from the user.

Step 11: Prompt the user to enter the following student details:

- Name
- Age
- Department
- CGPA

Step 12: Read the entered values using the Scanner object.

Step 13: Create a Student object by invoking the **parameterized constructor** and pass the user-entered values as arguments.

Step 14: Access the **public** data member department from the Student object and display it.

Step 15: Invoke the display() method to print all the student details.

Step 16: Close the Scanner object.

Step 17: Stop the program.

Program

Student.java

```
package mypackage;

public class Student
{
    String name;
    protected int age;
    public String department;
    private double cgpa;
    // Parameterized Constructor
    public Student(String name, int age, String department, double cgpa)
    {
        this.name = name;
        this.age = age;
        this.department = department;
        this.cgpa = cgpa;
    }
    // Public method to display student details
    public void display()
    {
        System.out.println("\n----- Student Details -----");
        System.out.println("Name      : " + name);
        System.out.println("Age      : " + age);
        System.out.println("Department : " + department);
        System.out.println("CGPA      : " + cgpa);
    }
}
```

PackageDemo.java

```
import java.util.Scanner;
import mypackage.Student;

public class PackageDemo
{
    public static void main(String args[])
    {
```

```

Scanner sc = new Scanner(System.in);
String name;
int age;
String department;
double cgpa;
System.out.print("Enter Student Name    : ");
name = sc.nextLine();
System.out.print("Enter Age            : ");
age = sc.nextInt();
sc.nextLine();    // Consume newline
System.out.print("Enter Department      : ");
department = sc.nextLine();
System.out.print("Enter CGPA          : ");
cgpa = sc.nextDouble();
// Constructor Invocation
Student obj = new Student(name, age, department, cgpa);
System.out.println("\nAccessing Public Data Member");
System.out.println("Student Dept : " + obj.department);
// Calling public method
obj.display();
sc.close();
}
}

```

Output

```

Enter Student Name    : Rahul
Enter Age            : 20
Enter Department      : Computer Science and Engineering
Enter CGPA          : 8.75

```

```

Accessing Public Data Member
Student Dept : Computer Science and Engineering

```

```

----- Student Details -----

```

Name : Rahul

Age : 20

Department : Computer Science and Engineering

CGPA : 8.75

Result

Thus, the program to create a user-defined package was implemented successfully. The output demonstrates package creation, constructor invocation, object initialization, user input handling, and controlled access to class members.

Ex.no:3.b**Implementation of Interface****Aim**

To develop a Java program to demonstrate the use of an interface with constants, abstract methods, default methods, static methods, and an interface reference variable, and to understand how these features are implemented and accessed in Java.

Algorithm

1. Start the program.
2. Declare an interface named Student.
3. Declare a constant COLLEGE.
 - The constant is automatically treated as public static final.
4. Declare two abstract methods in the interface:
 - `getDetails(String name, int age, String department)`
 - `displayDetails()`
5. Define a default method named `welcome()` inside the interface.
 - Display a welcome message for the Java Programming Laboratory.
6. Define a static method named `collegeInfo()` inside the interface.
 - Display the college name and department.
 - This method belongs to the interface and is called using the interface name.
7. Create a class named `StudentInfo` that implements the `Student` interface.
8. Declare three private data members in `StudentInfo`:
 - `name`
 - `age`
 - `department`
9. Implement the abstract method `getDetails()`.
 - Accept the student's name, age, and department as parameters.
 - Store these values in the instance variables using the `this` keyword.
10. Implement the abstract method `displayDetails()`.
 - Display the student's name.
 - Display the student's age.
 - Display the student's department.
 - Display the interface constant `COLLEGE`.
11. Create the main class `InterfaceDemo`.

12. Begin execution of the main() method.
13. Invoke the interface static method using the interface name:
 - Student.collegeInfo();
 - Display the college information.
14. Create an interface reference variable:
Student obj = new StudentInfo();
 - The interface reference points to the object of the implementing class.
15. Call the getDetails() method through the interface reference variable.
 - Pass the student's name, age, and department.
16. Call the interface default method welcome() using the interface reference variable.
 - Display the welcome message.
17. Call the implemented method displayDetails() using the interface reference variable.
 - Display all the student information.
18. Stop the program.

Program

```
interface Student
{
    String COLLEGE = "Velammal Institute of Technology";
    void getDetails(String name, int age, String department);
    void displayDetails();
    default void welcome()
    {
        System.out.println("Welcome to the Java Programming Laboratory");
    }
    static void collegeInfo()
    {
        System.out.println("College Name : " + COLLEGE);
        System.out.println("Department : Computer Science and Engineering");
    }
}
class StudentInfo implements Student
{
    private String name;
    private int age;
    private String department;
    public void getDetails(String name, int age, String department)
    {
        this.name = name;
        this.age = age;
        this.department = department;
    }
}
```

```

public void displayDetails()
{
    System.out.println("\n----- Student Details -----");
    System.out.println("Name      : " + name);
    System.out.println("Age       : " + age);
    System.out.println("Department : " + department);
    System.out.println("College   : " + COLLEGE);
}
}
public class InterfaceDemo
{
    public static void main(String args[])
    {
        Student.collegeInfo();
        System.out.println();
        Student obj = new StudentInfo();
        obj.getDetails("Arun", 20, "Computer Science and Engineering");
        obj.welcome();
        obj.displayDetails();
    }
}

```

Output

College Name : Velammal Institute of Technology
 Department : Computer Science and Engineering

Welcome to the Java Programming Laboratory

```

----- Student Details -----
Name      : Arun
Age       : 20
Department : Computer Science and Engineering
College   : Velammal Institute of Technology

```

Result

Thus, the program was successfully executed to demonstrate the use of an interface with abstract, default, and static methods, along with an interface reference variable.

Ex.No:4.a**Creation of User defined exception****Aim**

To develop a Java program to create and handle a **user-defined exception** by extending the Exception class and to demonstrate the use of throw, throws, try, and catch statements.

Algorithm

1. **Start** the program.
2. **Create** a user-defined exception class named InvalidAgeException by extending the Exception class.
3. **Define** a parameterized constructor in InvalidAgeException to receive the exception message.
4. Use super() to pass the message to the Exception class.
5. **Create** a class named Voting.
6. Define a method checkAge(int age) to check the eligibility of a person for voting.
7. Declare InvalidAgeException using the throws keyword.
8. Check whether the given age is less than 18.
9. If the age is less than 18, create and throw an InvalidAgeException object using the throw statement.
10. If the age is 18 or above, display that the person is eligible to vote.
11. **Create** the main class UserDefinedExceptionDemo.
12. Create an object of the Voting class.
13. Invoke the checkAge() method inside the try block.
14. If an InvalidAgeException occurs, catch it using the catch block.
15. Display the exception message using getMessage().
16. **Stop** the program.

Program

```
// User-defined Exception Class
class InvalidAgeException extends Exception
{
    // Constructor
    public InvalidAgeException(String message)
    {
        super(message);
    }
}

// Class to check voting eligibility
```

```

class Voting
{
    public void checkAge(int age) throws InvalidAgeException
    {
        if (age < 18)
        {
            throw new InvalidAgeException("Age is less than 18. Not eligible to vote.");
        }
        else
        {
            System.out.println("Eligible to vote.");
        }
    }
}

// Main Class
public class UserDefinedExceptionDemo
{
    public static void main(String args[])
    {
        Voting obj = new Voting();
        try
        {
            obj.checkAge(16);
        }
        catch (InvalidAgeException e)
        {
            System.out.println("Exception: " + e.getMessage());
        }
        System.out.println("Program terminated successfully.");
    }
}

```

Output

Exception: Age is less than 18. Not eligible to vote.
Program terminated successfully.

Result

Thus, the Java program was successfully executed to create and handle a user-defined exception using the `InvalidAgeException` class. The program demonstrated the use of inheritance, constructors, throw, throws, try, and catch statements for handling application-specific exceptions.

Ex.No:4.b**Creation of Multiple Threads****Aim**

To develop a Java program using multiple threads to generate a random number and create separate threads to calculate the square of an even number and the cube of an odd number, demonstrating multithreading.

Algorithm

1. Start the program.
2. Create a class named Even that extends the Thread class.
 - This thread is responsible for computing and displaying the square of a number.
3. Override the run() method in the Even class.
 - Write the code to calculate the square of the given number.
 - Display the square value.
4. Create a class named Odd that extends the Thread class.
 - This thread is responsible for computing and displaying the cube of a number.
5. Override the run() method in the Odd class.
 - Write the code to calculate the cube of the given number.
 - Display the cube value.
6. Create a main class named MultiThreadDemo.
7. Generate a random number using the Random class.
8. Check whether the generated number is even or odd.
9. If the generated number is even:
 - Create an object of the Even thread.
 - Pass the generated number to the thread.
 - Start the thread using the start() method.
 - The thread calculates and displays the square of the number.
10. If the generated number is odd:
 - Create an object of the Odd thread.
 - Pass the generated number to the thread.
 - Start the thread using the start() method.
 - The thread calculates and displays the cube of the number.
11. Use the sleep() method to temporarily suspend the executing thread for a specified period.
12. The JVM schedules the created thread for execution, demonstrating multithreading and concurrent execution.
13. Wait for the thread to complete its execution.
14. Stop the program.

Program

```
import java.util.Random;

// Thread to calculate square of an even number
class Even extends Thread
{
    int number;

    Even(int number)
```

```

    {
        this.number = number;
    }

    public void run()
    {
        System.out.println("Even Thread");
        System.out.println("Number : " + number);
        System.out.println("Square : " + (number * number));

        try
        {
            Thread.sleep(1000);
        }
        catch (InterruptedException e)
        {
            System.out.println(e);
        }
    }
}

// Thread to calculate cube of an odd number
class Odd extends Thread
{
    int number;

    Odd(int number)
    {
        this.number = number;
    }

    public void run()
    {
        System.out.println("Odd Thread");
        System.out.println("Number : " + number);
        System.out.println("Cube : " + (number * number * number));

        try
        {
            Thread.sleep(1000);
        }
        catch (InterruptedException e)
        {
            System.out.println(e);
        }
    }
}

// Main class
public class MultiThreadDemo
{
    public static void main(String args[])

```

```

{
    Random r = new Random();

    int number = r.nextInt(100);

    System.out.println("Random Number Generated : " + number);

    if (number % 2 == 0)
    {
        Even evenThread = new Even(number);
        evenThread.start();
    }
    else
    {
        Odd oddThread = new Odd(number);
        oddThread.start();
    }

    System.out.println("Main Thread is executing...");
}
}

```

Output

Even Number

Random Number Generated : 8
 Main Thread is executing...
 Even Thread
 Number : 8
 Square : 64

Odd Number

Random Number Generated : 7
 Main Thread is executing...
 Odd Thread
 Number : 7
 Cube : 343

Result

Thus, the Java program was successfully executed to demonstrate multithreading using the Thread class. A random number was generated, and based on whether the number was even or odd, a separate thread was created to calculate its square or cube.

Ex.No:5**String Handling****Aim:**

To write a Java program to perform various string handling operations such as finding the length of a string, concatenation, comparison, substring extraction, character searching, and changing the case of characters.

Algorithm

1. Start the program.
2. Read two strings from the user.
3. Find and display the length of each string using length().
4. Concatenate the two strings using concat().
5. Compare the strings using equals() and equalsIgnoreCase().
6. Convert the strings into uppercase and lowercase using toUpperCase() and toLowerCase().
7. Extract a substring using substring().
8. Search for a character or substring using indexOf().
9. Display all the results.
10. Stop the program.

Program

```
import java.util.Scanner;
```

```
public class StringHandling {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter first string: ");
        String str1 = sc.nextLine();

        System.out.print("Enter second string: ");
        String str2 = sc.nextLine();

        System.out.println("\nLength of first string: " + str1.length());
        System.out.println("Length of second string: " + str2.length());

        System.out.println("Concatenated String: " + str1.concat(" " + str2));

        System.out.println("Strings are equal: " + str1.equals(str2));
        System.out.println("Strings are equal ignoring case: " + str1.equalsIgnoreCase(str2));

        // Case conversion
        System.out.println("First string in uppercase: " + str1.toUpperCase());
        System.out.println("First string in lowercase: " + str1.toLowerCase());

        // Substring
        if (str1.length() >= 3) {
            System.out.println("Substring of first string: "
                + str1.substring(0, 3));
        }
    }
}
```

```

// Character at a particular position
if (str1.length() > 0) {
    System.out.println("First character: " + str1.charAt(0));
}

// Searching
System.out.println("Position of 'a' in first string: "
    + str1.indexOf('a'));

    sc.close();
}
}

```

Sample Input

Enter first string: Java

Enter second string: Programming

Sample Output

Length of first string: 4

Length of second string: 11

Concatenated String: Java Programming

Strings are equal: false

Strings are equal ignoring case: false

First string in uppercase: JAVA

First string in lowercase: java

Substring of first string: Jav

First character: J

Position of 'a' in first string: 1

Result

Thus, the Java program to perform various **String Handling operations** such as concatenation, comparison, case conversion, substring extraction, character access, and searching was successfully executed.

Ex.No:6.a

Event Handling Using AWT

Aim:

To write a Java program to demonstrate **event handling using AWT controls** by handling button click events.

Algorithm

1. Start the program.
2. Create a class that extends Frame and implements the ActionListener interface.
3. Create AWT controls such as Label, TextField, and Button.
4. Add the controls to the frame.
5. Register the button with the ActionListener.
6. Implement the actionPerformed() method.
7. When the button is clicked, read the text from the text field.
8. Display the entered text in the label.
9. Set the frame size and make it visible.
10. Stop the program.

Program

```
import java.awt.*;
import java.awt.event.*;

public class AWTEventHandling extends Frame implements ActionListener {

    Label label1, label2;
    TextField textField;
    Button button;

    AWTEventHandling() {
        // Create AWT controls
        label1 = new Label("Enter your name:");
        textField = new TextField(20);
        button = new Button("Submit");
        label2 = new Label(" ");

        // Set layout
        setLayout(new FlowLayout());

        // Add controls to frame
        add(label1);
        add(textField);
        add(button);
        add(label2);

        // Register event listener
        button.addActionListener(this);

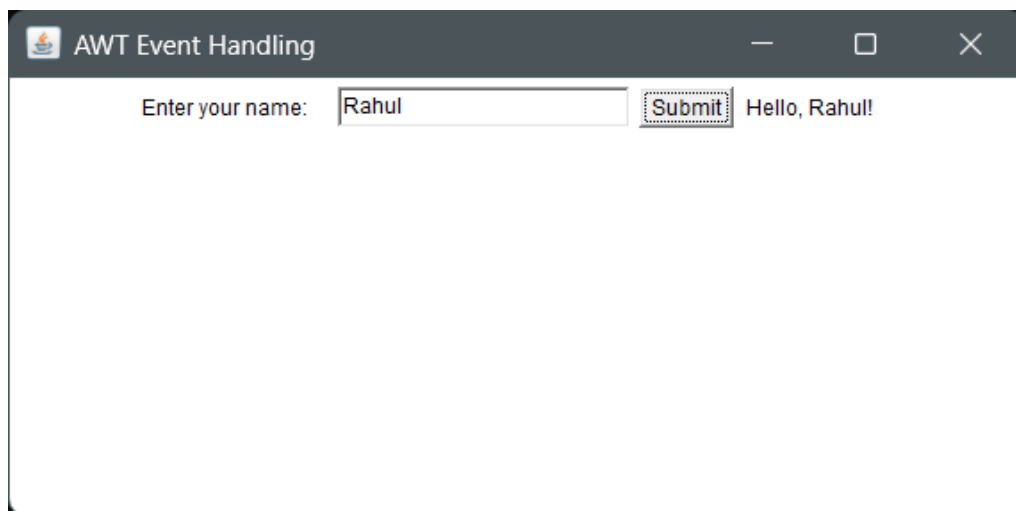
        // Frame settings
        setTitle("AWT Event Handling");
        setSize(400, 200);
        setVisible(true);
    }
}
```

```
// Close the window
addWindowListener(new WindowAdapter() {
    public void windowClosing(WindowEvent e) {
        dispose();
    }
});
}

// Event handling method
public void actionPerformed(ActionEvent e) {
    String name = textField.getText();
    label2.setText("Hello, " + name + "!");
}

public static void main(String[] args) {
    new AWTEventHandling();
}
}
```

Sample Output



Result

Thus, the Java program to demonstrate AWT event handling using the ActionListener interface and(ActionEvent) class was successfully executed.

Ex.No:6.b. Handling Key and Mouse Events Using AWT

Aim

To write a Java program to demonstrate **keyboard and mouse event handling** using AWT event classes and listener interfaces.

Algorithm

1. Start the program.
2. Create a class that extends Frame.
3. Implement the KeyListener and MouseListener interfaces.
4. Create an AWT Label to display the generated events.
5. Register the frame with the keyboard and mouse listeners.
6. Implement the methods of KeyListener to handle:
 - Key pressed
 - Key released
 - Key typed
7. Implement the methods of MouseListener to handle:
 - Mouse clicked
 - Mouse pressed
 - Mouse released
 - Mouse entered
 - Mouse exited
8. Display the corresponding event message on the label.
9. Set the frame size and make it visible.
10. Stop the program.

Program

```
import java.awt.*;
import java.awt.event.*;

public class KeyMouseEventDemo extends Frame implements KeyListener, MouseListener {

    Label label;

    KeyMouseEventDemo() {

        label = new Label("Perform a keyboard or mouse action");
        label.setFont(new Font("Arial", Font.BOLD, 16));

        setLayout(new FlowLayout());
        add(label);

        // Register listeners
        addKeyListener(this);
        addMouseListener(this);

        setTitle("Key and Mouse Event Handling");
        setSize(500, 300);
        setVisible(true);
    }
}
```

```

// Close the window
addWindowListener(new WindowAdapter() {
    public void windowClosing(WindowEvent e) {
        dispose();
    }
});

// Enable keyboard focus
setFocusable(true);
requestFocus();
}

// KeyListener methods
public void keyPressed(KeyEvent e) {
    label.setText("Key Pressed: " + e.getKeyChar());
}

public void keyReleased(KeyEvent e) {
    label.setText("Key Released: " + e.getKeyChar());
}

public void keyTyped(KeyEvent e) {
    label.setText("Key Typed: " + e.getKeyChar());
}

// MouseListener methods
public void mouseClicked(MouseEvent e) {
    label.setText("Mouse Clicked at (" +
        e.getX() + ", " + e.getY() + ")");
}

public void mousePressed(MouseEvent e) {
    label.setText("Mouse Pressed");
}

public void mouseReleased(MouseEvent e) {
    label.setText("Mouse Released");
}

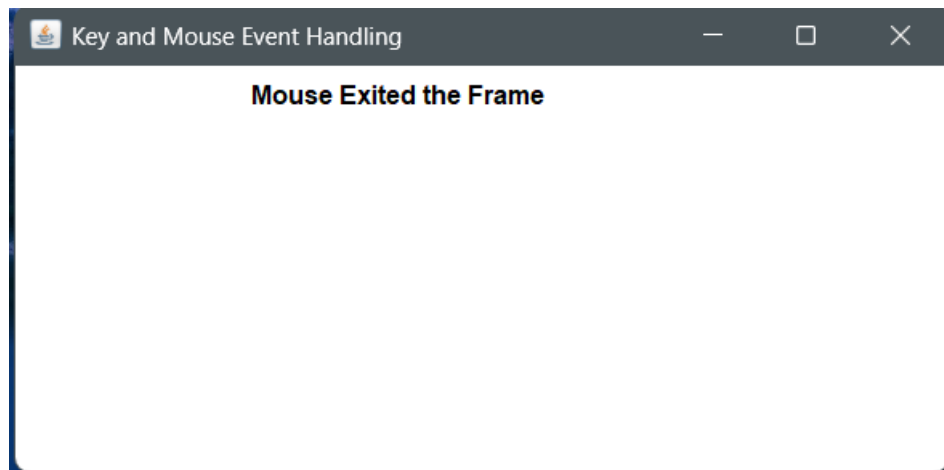
public void mouseEntered(MouseEvent e) {
    label.setText("Mouse Entered the Frame");
}

public void mouseExited(MouseEvent e) {
    label.setText("Mouse Exited the Frame");
}

public static void main(String[] args) {
    new KeyMouseEventDemo();
}
}

```

Sample output



Result

Thus, the Java program to demonstrate handling of keyboard and mouse events using AWT was successfully executed.

Ex.No:7

File I/O Operations

Aim

To write and execute a Java program to demonstrate various **File I/O operations**, such as creating a file, writing data, appending data, reading data, displaying file information, copying a file, and deleting a file.

Algorithm

1. Start the program.
2. Create a File object for student.txt.
3. Create the file using createNewFile().
4. Write data into the file using FileWriter.
5. Append additional data using FileWriter in append mode.
6. Read the file using FileReader.
7. Display file information using the File class.
8. Copy the file using FileInputStream and FileOutputStream.
9. Delete the copied file using delete().
10. Display the result of each operation.
11. Stop the program.

Program

```
import java.io.*;
public class FileOperations {
    public static void main(String[] args) {
        String fileName = "student.txt";
        String copyFileName = "student_copy.txt";

        // 1. CREATE FILE
        File file = new File(fileName);

        try {
            if (file.createNewFile()) {
                System.out.println("1. File created successfully.");
            } else {
                System.out.println("1. File already exists.");
            }
        } catch (IOException e) {
            System.out.println("Error while creating file.");
        }

        // 2. WRITE DATA INTO FILE
        try {
            FileWriter writer = new FileWriter(fileName);

            writer.write("Name: Arun\n");
```

```

writer.write("Course: Java Programming\n");
writer.write("Marks: 85\n");

writer.close();

System.out.println("2. Data written successfully.");

} catch (IOException e) {
    System.out.println("Error while writing file.");
}

// 3. APPEND DATA
try {
    FileWriter writer = new FileWriter(fileName, true);

    writer.write("Grade: A\n");

    writer.close();

    System.out.println("3. Data appended successfully.");

} catch (IOException e) {
    System.out.println("Error while appending file.");
}

// 4. READ FILE
System.out.println("\n4. File Contents:");

try {
    FileReader reader = new FileReader(fileName);

    int ch;

    while ((ch = reader.read()) != -1) {
        System.out.print((char) ch);
    }

    reader.close();

} catch (IOException e) {
    System.out.println("Error while reading file.");
}

// 5. DISPLAY FILE INFORMATION
System.out.println("\n5. File Information:");

if (file.exists()) {
    System.out.println("File Name    : " + file.getName());
    System.out.println("Absolute Path : " + file.getAbsolutePath());
}

```

```

        System.out.println("File Size      : " + file.length() + " bytes");
        System.out.println("Readable      : " + file.canRead());
        System.out.println("Writable      : " + file.canWrite());
    }

    // 6. COPY FILE
    try {
        FileInputStream input =
            new FileInputStream(fileName);

        FileOutputStream output =
            new FileOutputStream(copyFileName);

        int data;

        while ((data = input.read()) != -1) {
            output.write(data);
        }

        input.close();
        output.close();

        System.out.println("\n6. File copied successfully.");

    } catch (IOException e) {
        System.out.println("Error while copying file.");
    }

    // 7. DELETE COPIED FILE
    File copyFile = new File(copyFileName);

    if (copyFile.delete()) {
        System.out.println("7. Copied file deleted successfully.");
    } else {
        System.out.println("7. File could not be deleted.");
    }
}
}

```

Sample Output

1. File created successfully.
2. Data written successfully.
3. Data appended successfully.
4. File Contents:
 Name: Arun
 Course: Java Programming
 Marks: 85
 Grade: A

5. File Information:

File Name : student.txt

Absolute Path : C:\Java\student.txt

File Size : 64 bytes

Readable : true

Writable : true

6. File copied successfully.

7. Copied file deleted successfully.

Result

Thus, the Java program was successfully executed and the required **File I/O operations** such as creating, writing, reading, appending, copying, and deleting files were demonstrated.

Ex.No:8

Swing Application with Menus

Aim

To write a Java program to develop a **Swing application with a menu bar, menus, and menu items** and handle menu item events.

Algorithm

1. Start the program.
2. Create a JFrame for the Swing application.
3. Create a JMenuBar.
4. Create menus such as **File** and **Help**.
5. Create menu items such as **New**, **Open**, **Save**, **Exit**, and **About**.
6. Add the menu items to their respective menus.
7. Add the menus to the menu bar.
8. Set the menu bar to the frame.
9. Register an ActionListener for the menu items.
10. Perform the appropriate action when a menu item is selected.
11. Display messages using JOptionPane.
12. Stop the program.

Program

```
import javax.swing.*;
import java.awt.event.*;

public class SwingMenuApplication extends JFrame implements ActionListener {

    JMenuBar menuBar;
    JMenu fileMenu, helpMenu;
    JMenuItem newItem, openItem, saveItem, exitItem, aboutItem;

    SwingMenuApplication() {

        // Create menu bar
        menuBar = new JMenuBar();

        // Create menus
        fileMenu = new JMenu("File");
        helpMenu = new JMenu("Help");

        // Create menu items
        newItem = new JMenuItem("New");
        openItem = new JMenuItem("Open");
        saveItem = new JMenuItem("Save");
        exitItem = new JMenuItem("Exit");
        aboutItem = new JMenuItem("About");

        // Add menu items to File menu
        fileMenu.add(newItem);
        fileMenu.add(openItem);
```

```

fileMenu.add(saveItem);

// Separator
fileMenu.addSeparator();

fileMenu.add(exitItem);

// Add menu item to Help menu
helpMenu.add(aboutItem);

// Add menus to menu bar
menuBar.add(fileMenu);
menuBar.add(helpMenu);

// Set menu bar
setJMenuBar(menuBar);

// Register event listeners
newItem.addActionListener(this);
openItem.addActionListener(this);
saveItem.addActionListener(this);
exitItem.addActionListener(this);
aboutItem.addActionListener(this);

// Frame settings
setTitle("Swing Menu Application");
setSize(500, 300);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
setLocationRelativeTo(null);
setVisible(true);
}

public void actionPerformed(ActionEvent e) {

    if (e.getSource() == newItem) {
        JOptionPane.showMessageDialog(this,
            "New File Selected");
    }

    else if (e.getSource() == openItem) {
        JOptionPane.showMessageDialog(this,"Open File Selected");
    }

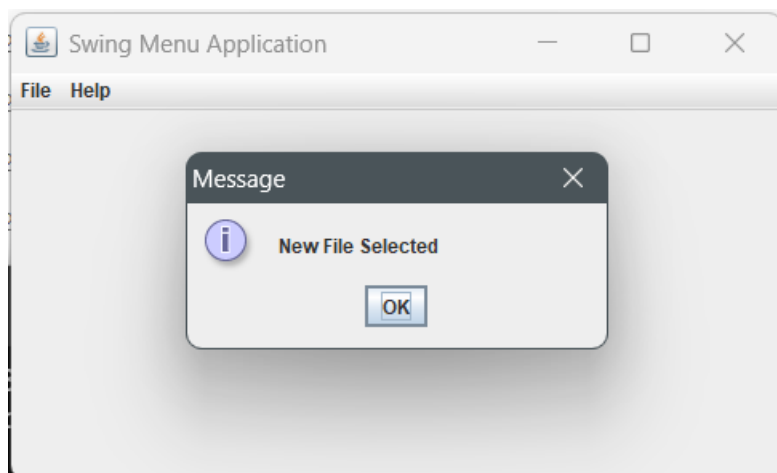
    else if (e.getSource() == saveItem) {
        JOptionPane.showMessageDialog(this,"File Saved Successfully");
    }

    else if (e.getSource() == aboutItem) {
        JOptionPane.showMessageDialog(this,"Swing Menu Application\nJava
Programming");
    }
}

```

```
        else if (e.getSource() == exitItem) {  
            System.exit(0);  
        }  
    }  
  
    public static void main(String[] args) {  
        new SwingMenuApplication();  
    }  
}
```

Sample Output



Result

Thus, the Java program to develop a Swing application with menus and menu items and handle menu events using ActionListener was successfully executed.

Ex.No:9

Aim

Database Connections and Operations Using Oracle

To develop a Java application using **JDBC** to establish a connection with an **Oracle Database** and perform basic database operations such as **INSERT, UPDATE, DELETE, and SELECT**.

Algorithm

- Step 1: Start the program.
- Step 2: Load the Oracle JDBC driver.
- Step 3: Establish a connection with the Oracle database using DriverManager.
- Step 4: Create a PreparedStatement object.
- Step 5: Insert student records into the student table.
- Step 6: Update the marks of a student.
- Step 7: Delete a student record.
- Step 8: Execute a SELECT query to retrieve the student records.
- Step 9: Display the retrieved records using ResultSet.
- Step 10: Close the database resources and connection.
- Step 11: Stop the program.

Program

```
import java.sql.*;

public class OracleDatabaseOperations {

    // Oracle Database connection details
    static final String URL ="jdbc:oracle:thin:@localhost:1521:XE";

    static final String USER = "system";
    static final String PASSWORD = "oracle";

    public static void main(String[] args) {

        Connection con = null;
        PreparedStatement pstmt = null;
        ResultSet rs = null;

        try {
            // Step 1: Load Oracle JDBC Driver
            Class.forName("oracle.jdbc.driver.OracleDriver");

            // Step 2: Establish Database Connection
            con = DriverManager.getConnection(URL, USER, PASSWORD);

            System.out.println("Oracle Database Connected Successfully!");

            String insertSQL ="INSERT INTO student " + "(rollno, name, department, marks) " +
                "VALUES (?, ?, ?, ?)";

            pstmt = con.prepareStatement(insertSQL);
```

```

// Insert first record
pstmt.setInt(1, 101);
pstmt.setString(2, "Arun");
pstmt.setString(3, "CSE");
pstmt.setInt(4, 85);

pstmt.executeUpdate();

// Insert second record
pstmt.setInt(1, 102);
pstmt.setString(2, "Priya");
pstmt.setString(3, "IT");
pstmt.setInt(4, 90);

pstmt.executeUpdate();

System.out.println("Records inserted successfully.");

pstmt.close();

String updateSQL = "UPDATE student SET marks = ? " + "WHERE rollno = ?";
pstmt = con.prepareStatement(updateSQL);
pstmt.setInt(1, 95);
pstmt.setInt(2, 101);

pstmt.executeUpdate();
System.out.println("Record updated successfully.");
pstmt.close();

String deleteSQL = "DELETE FROM student WHERE rollno = ?";
pstmt = con.prepareStatement(deleteSQL);

pstmt.setInt(1, 102);

pstmt.executeUpdate();

System.out.println("Record deleted successfully.");

pstmt.close();
String selectSQL = "SELECT rollno, name, department, marks " + "FROM student";
pstmt = con.prepareStatement(selectSQL);
rs = pstmt.executeQuery();

System.out.println("\nStudent Details");
System.out.println(
    "-----");

System.out.printf("%-10s %-15s %-15s %-10s\n",
    "Roll No", "Name", "Department", "Marks");

System.out.println(
    "-----");

```

```

while (rs.next()) {

    System.out.printf("%-10d %-15s %-15s %-10d%n",
        rs.getInt("rollno"),
        rs.getString("name"),
        rs.getString("department"),
        rs.getInt("marks"));

}

System.out.println("-----");

}
catch (ClassNotFoundException e) {

    System.out.println("Oracle JDBC Driver not found.");
    e.printStackTrace();
}
catch (SQLException e) {

    System.out.println(
        "Oracle Database Error.");

    e.printStackTrace();
}
finally {

    // Close resources
    try {

        if (rs != null)
            rs.close();

        if (pstmt != null)
            pstmt.close();

        if (con != null)
            con.close();

        System.out.println(
            "\nOracle Database Connection Closed.");

    }
    catch (SQLException e) {
        e.printStackTrace();
    }
}
}
}

```

```

CREATE TABLE student ( rollno NUMBER(5) PRIMARY KEY, name VARCHAR2(50),
department VARCHAR2(30), marks NUMBER(3) );

```

Sample Output

Oracle Database Connected Successfully!
Records inserted successfully.
Record updated successfully.
Record deleted successfully.

Student Details

Roll No	Name	Department	Marks
101	Arun	CSE	95

Oracle Database Connection Closed.

Result

The Java program was successfully developed using JDBC with Oracle Database to establish a database connection and perform INSERT, UPDATE, DELETE, and SELECT operations successfully.

Ex.No:10. a

Aim

ArrayList and List Interface

To write a Java program to demonstrate the use of the **List interface** and **ArrayList class** for storing and manipulating a collection of elements.

Algorithm

- Step 1:** Start the program.
- Step 2:** Create an ArrayList using the List interface.
- Step 3:** Add student names to the list.
- Step 4:** Display all elements.
- Step 5:** Access an element using its index.
- Step 6:** Modify an element using the set() method.
- Step 7:** Remove an element using the remove() method.
- Step 8:** Search for an element using contains().
- Step 9:** Display the size of the list.
- Step 10:** Traverse the list using an enhanced for loop.
- Step 11:** Stop the program.

Program

```
import java.util.ArrayList;
import java.util.List;

public class ArrayListDemo {

    public static void main(String[] args) {

        // Creating ArrayList using List interface
        List<String> students = new ArrayList<>();

        // Adding elements
        students.add("Arun");
        students.add("Priya");
        students.add("Kavin");
        students.add("Divya");
        students.add("Rahul");

        System.out.println("Student List:");
        System.out.println(students);

        // Accessing an element
        System.out.println("\nStudent at index 2: "+ students.get(2));

        // Adding element at a particular position
        students.add(2, "Anitha");

        System.out.println("\nAfter adding Anitha:");
        System.out.println(students);
    }
}
```

```

// Modifying an element
students.set(1, "Priyanka");

System.out.println("\nAfter modifying the second element:");
System.out.println(students);

// Removing an element
students.remove("Rahul");

System.out.println("\nAfter removing Rahul:");
System.out.println(students);

// Searching an element
if (students.contains("Kavin")) {
    System.out.println("\nKavin is present in the list.");
} else {
    System.out.println("\nKavin is not present in the list.");
}

// Finding size
System.out.println("\nNumber of students: "+ students.size());

// Traversing the list
System.out.println("\nTraversing the ArrayList:");

for (String student : students) {
    System.out.println(student);
}
}
}

```

Sample output

Student List:

[Arun, Priya, Kavin, Divya, Rahul]

Student at index 2: Kavin

After adding Anitha:

[Arun, Priya, Anitha, Kavin, Divya, Rahul]

After modifying the second element:

[Arun, Priyanka, Anitha, Kavin, Divya, Rahul]

After removing Rahul:

[Arun, Priyanka, Anitha, Kavin, Divya]

Kavin is present in the list.

Number of students: 5

Traversing the ArrayList:

Arun

Priyanka

Anitha

Kavin

Divya

Result

Thus, the Java program was successfully executed to demonstrate the **List interface and ArrayList class**, including adding, accessing, modifying, removing, searching, and traversing elements.

Ex.No:10.b

Map Interface

Aim

To write a Java program to demonstrate the use of the **Map interface** and its implementation using **HashMap** for storing and manipulating key-value pairs.

Algorithm

- Step 1:** Start the program.
- Step 2:** Create a Map using the HashMap class.
- Step 3:** Add student roll numbers and names using the put() method.
- Step 4:** Display the map.
- Step 5:** Retrieve a student's name using the get() method.
- Step 6:** Update the value associated with an existing key.
- Step 7:** Check whether a particular key exists using containsKey().
- Step 8:** Remove an entry using remove().
- Step 9:** Traverse the map using entrySet().
- Step 10:** Display the number of entries using size().
- Step 11:** Stop the program.

Program

```
import java.util.HashMap;
import java.util.Map;

public class MapInterfaceDemo {

    public static void main(String[] args) {

        // Creating Map using HashMap
        Map<Integer, String> students = new HashMap<>();

        // Adding key-value pairs
        students.put(101, "Arun");
        students.put(102, "Priya");
        students.put(103, "Kavin");
        students.put(104, "Divya");
        students.put(105, "Rahul");

        System.out.println("Student Map:");
        System.out.println(students);

        // Accessing value using key
        System.out.println("\nStudent with Roll No 103: "+ students.get(103));

        // Updating a value
        students.put(103, "Karthik");

        System.out.println("\nAfter updating Roll No 103:");
        System.out.println(students);
    }
}
```

```

// Checking whether a key exists
if (students.containsKey(102)) {
    System.out.println("\nRoll No 102 is present in the map.");
}

// Checking whether a value exists
if (students.containsValue("Divya")) {
    System.out.println(
        "Student Divya is present in the map.");
}

// Removing an entry
students.remove(105);

System.out.println("\nAfter removing Roll No 105:");
System.out.println(students);

// Displaying size
System.out.println("\nNumber of students: " + students.size());

// Traversing using entrySet()
System.out.println("\nStudent Details:");

for (Map.Entry<Integer, String> entry: students.entrySet()) {
    System.out.println(
        "Roll No: " + entry.getKey()
        + ", Name: " + entry.getValue());
}
}
}

```

Sample Output

Student Map:

{101=Arun, 102=Priya, 103=Kavin, 104=Divya, 105=Rahul}

Student with Roll No 103: Kavin

After updating Roll No 103:

{101=Arun, 102=Priya, 103=Karthik, 104=Divya, 105=Rahul}

Roll No 102 is present in the map.

Student Divya is present in the map.

After removing Roll No 105:

{101=Arun, 102=Priya, 103=Karthik, 104=Divya}

Number of students: 4

Student Details:

Roll No: 101, Name: Arun
Roll No: 102, Name: Priya
Roll No: 103, Name: Karthik
Roll No: 104, Name: Divya

Result

Thus, the Java program was successfully executed to demonstrate the Map interface using HashMap, including adding, accessing, updating, searching, removing, and traversing key-value pairs.